CruiseStream StudiosCover Page**Day 2**

MYNDNOVA SOLUTIONS ◆ CRUISESTREAM STUDIOS

Day 2 — GenAI, RAG & Vector Search

Foundation Models · Embeddings · Anti-Hallucination RAG

3

Modules

3

Labs

6

Hours

Dr. Yogesh Ghorpade

Solutions Architect — AI & Data ◆ MyndNova Solutions

linkedin.com/in/yogesh-ghorpade

Today's Agenda

4

LLMs & GenAI

Foundation Models
Summarization, Q&A
Genre Classification

5

Vector Search

Embeddings & vectors
Semantic search
Cosine similarity

6

RAG Pipeline

Retrieve + Augment + Generate
Anti-hallucination
Grounded answers

MODULE 4 / 03

Databricks Foundation Models

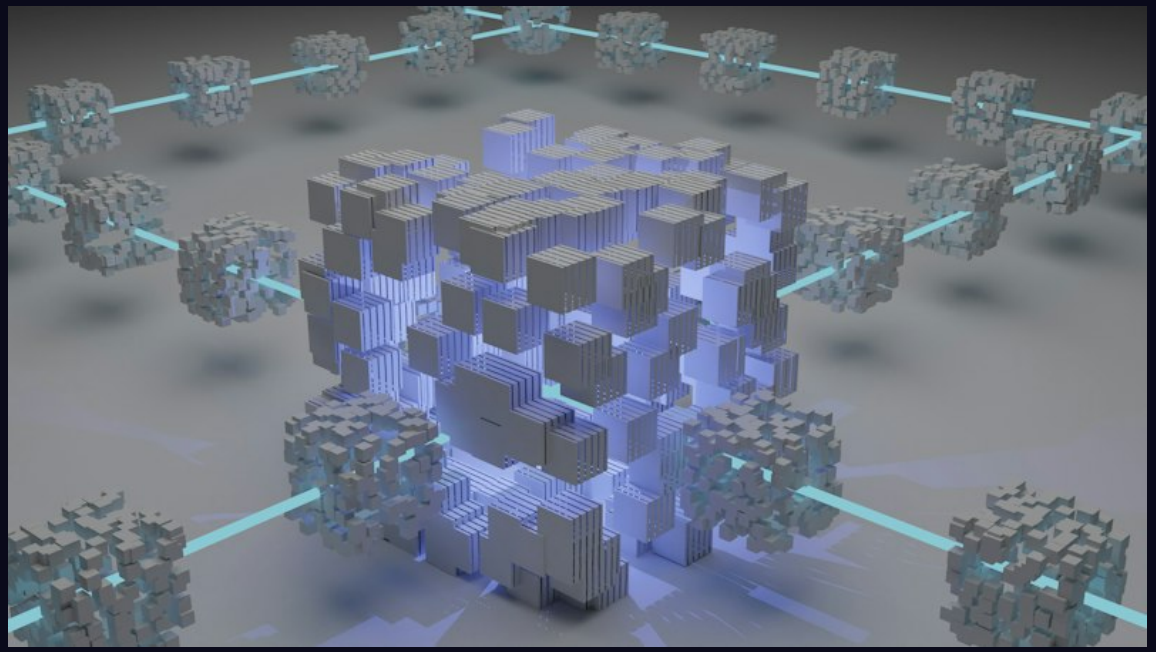
Text: Llama 3.3 70B

70B params, PAT-free, summarization/Q&A/classification

Embeddings: GTE Large

1024-dim vectors, PAT-free, semantic search

```
import mlflow.deployments
client = mlflow.deployments.get_deploy_client("databricks")
response = client.predict_endpoint(
   endpoint="databricks-meta-llama-3-3-70b-instruct",
   inputs={"messages":[{"role":"user",
       "content":"Summarize Top Gun"}],
       "temperature":0.3,"max_tokens":200})
```



■ **KEY LOGIC**`get_deploy_client("databricks")` creates PAT-free connection. Temperature 0.3 = focused.

MODULE 4 / 04

GenAI Task 1: Summarization

```
def summarize_movie(movie_title, plot_text):
   prompt = f"""Summarize {movie_title} in exactly 2 sentences.
   Plot: {plot_text}"""
   response = client.predict_endpoint(
       endpoint="databricks-meta-llama-3-3-70b-instruct",
       inputs={"messages":[{"role":"user","content":prompt}],
               "temperature":0.3,"max_tokens":150})
   return response["choices"][0]["message"]["content"]

# Test: Summarize Top Gun: Maverick
# Output: "A veteran naval aviator returns to train
#  a new generation for a perilous mission."
```

■ **KEY LOGIC**"Exactly 2 sentences" constrains output. `temperature: 0.3` ensures consistency. `max_tokens: 150` prevents over-generation.

Use Case: CruiseStream UI needs 2-sentence movie card descriptions. GenAI generates 12 in seconds.

MODULE 4 / 05

GenAI Task 2: Q&A

```
def movie_qa(question, movie_context=""):
   prompt = f"""You are a movie expert.
   Context: {movie_context}
   Question: {question}"""
   response = client.predict_endpoint(
       endpoint="databricks-meta-llama-3-3-70b-instruct",
       inputs={"messages":[{"role":"user","content":prompt}],
               "temperature":0.7,"max_tokens":200})
   return response["choices"][0]["message"]["content"]

# movie_qa("What genre is Rain Man?")  -> "Drama"
# movie_qa("Which has highest IMDb?")  -> "Rain Man at 8.0/10"
# movie_qa("What is Top Gun about?")   -> "A young naval aviator..."
```

■ **KEY LOGIC** `temperature: 0.7` allows conversational answers. System prompt "You are a movie expert" sets AI persona.

MODULE 4 / 06

GenAI Task 3: Genre Classification

```
def classify_genre(plot_summary):
   prompt = f"""Classify: Action, Drama, Sci-Fi, Thriller.
   Reply with ONLY the genre name.
   Plot: {plot_summary}"""
   response = client.predict_endpoint(
       inputs={"messages":[{"role":"user","content":prompt}],
               "temperature":0.1,"max_tokens":10})
   return response["choices"][0]["message"]["content"]

# "soldier relives day vs aliens" -> Sci-Fi ✓
# "sports agent moral awakening" -> Drama ✓
# "elite pilot trains aviators"  -> Action ✓
# 5/5 correct!
```

■ **KEY LOGIC** `temperature: 0.1` = deterministic. `max_tokens: 10` = single word. "ONLY the genre name" eliminates explanations.

LAB 4 / 07

Hands-On Lab

Lab 4: GenAI with Foundation Models

- Use `mlflow.deployments` to call Llama 3.3 70B
- Summarize Top Gun: Maverick in 2 sentences
- Ask AI movie questions (genre, cast, plot)
- Classify 5 movies by genre from plot alone — verify 5/5
- Experiment: temperature 0.1 (focused) vs 0.7 (creative)

Result: 5/5 genre classification. Temperature 0.1 = deterministic; 0.7 = creative.

MODULE 5 / 08

Embeddings — Text to Numbers

```
response = client.predict_endpoint(
   endpoint="databricks-gte-large-en",
   inputs={"input":["A fighter pilot trains aviators",
           "A savant and his brother road trip"]})
embeddings = [e["embedding"] for e in response["data"]]
# Each = list of 1024 floats
```

■ **KEY LOGIC**GTE Large converts each sentence into 1024 float numbers. Similar plots produce similar number patterns.

MODULE 5 / 09

Semantic vs Keyword Search

Keyword Search

```
Search: "brothers road trip discovery"
Result: 0 matches
(No movie has these exact words)
```

Fails when words don't match exactly

Semantic Search

```
def cosine_sim(a, b):
   return dot(a,b) / (norm(a) * norm(b))

query_emb = get_embedding("brothers road trip")
Result: Rain Man (0.73)
```

Finds by meaning, not exact words

■ **KEY LOGIC** `cosine_sim(a, b)` = dot product / (magnitude × magnitude). Returns 0-1: 1.0 = identical meaning. Threshold 0.6 filters weak matches.

LAB 5 / 10

Hands-On Lab

Lab 5: Vector Search Pipeline

- Create knowledge base from 12 movie plot summaries
- Generate 1024-dim GTE Large embeddings for each plot
- Build semantic search function using cosine similarity
- Compare keyword vs semantic search side-by-side
- Test: "time travel war" -> finds Edge of Tomorrow

Key finding: Semantic search found movies that keyword search completely missed.

MODULE 6 / 11

RAG Pipeline — 3 Steps

MODULE 6 / 12

RAG Implementation

```
def rag_answer(question):
   # Step 1: RETRIEVE
   query_emb = get_embedding(question)
   results = [(m, cosine_sim(query_emb, m["embedding"]))
              for m in movie_knowledge_base]
   top_movies = sorted(results, key=lambda x: x[1], reverse=True)[:3]

   # Step 2: AUGMENT
   context = "\n".join([f"Title: {m['title']}\nPlot: {m['summary']}"
       for m, _ in top_movies])

   # Step 3: GENERATE
   prompt = f"""Answer using ONLY the data below.
   If info missing, say "I don't have enough info."
   Movie Data: {context}\nQuestion: {question}"""
   return call_llm(prompt, temperature=0.3)
```

■ **KEY LOGIC**Step 1: cosine_sim ranks, takes top 3. Step 2: builds context from real data. Step 3: "Answer using ONLY the data below" prevents hallucination.

MODULE 6 / 13

Anti-Hallucination in Action

Plain GenAI (No RAG)

```
movie_qa("Budget of Top Gun: Maverick?")
-> "Approximately $170 million..."
(HALLUCINATED — not in DB)
```

RAG (Grounded)

```
rag_answer("Budget of Top Gun: Maverick?")
-> "I don't have enough info."
(HONEST — data not available)
```

■ **KEY LOGIC** "If info is missing, say 'I don't have enough info'" is the anti-hallucination guardrail. 3/3 edge-case tests passed.

LAB 6 / 14

Hands-On Lab

Lab 6: Build the RAG Pipeline

- Build `rag_answer()`: retrieve → augment → generate
- Test with real movie questions ("What is Top Gun about?")
- Test anti-hallucination: budgets, directors — not in database
- Compare GenAI vs RAG answers side-by-side
- Verify RAG provides source citations for every answer

Result: 3/3 anti-hallucination tests passed. RAG refused when data unavailable.

MODULE 6 / 15

Real-World Use Cases

Viewer Q&A Bot

"Which movie is about time travel?" RAG retrieves Edge of Tomorrow, augments with real plot, generates grounded answer.

Content Discovery

"I want something about brothers and road trips." Keyword = 0 results. Semantic search finds Rain Man (0.73).

Compliance Check

"Budget of Rain Man?" RAG checks KB, finds no budget data, responds: "I don't have enough info." Prevents fabrication.

Auto-Tagging

New movies arrive without genre. GenAI reads plot, classifies: Action/Drama/Sci-Fi/Thriller — 5/5 accuracy.

Day 2 Recap

Module 4

GenAI Tasks

Llama 3.3 70B (PAT-free)
Summarize: temp 0.3
Q&A: temp 0.7
Classify: temp 0.1 → 5/5

Module 5

Vector Search

GTE Large 1024-dim
cosine_sim = dot/norm
Semantic beats keyword

Module 6

RAG Pipeline

Retrieve top 3 + augment
"Answer ONLY from data"
Anti-hallucination 3/3

Tomorrow: Build a multi-tool AI Agent with SQL, semantic search, and box office calculator. Evaluate with MLflow.

Dr. Yogesh Ghorpade
MyndNova Solutions ◆ Databricks Certified
linkedin.com/in/yogesh-ghorpade